AI-Assisted Development – Exam

Expense Tracker & Budget Analyzer

Project Documentation

Student: Valentina Antimova

AI Development Tool: Claude Code (Visual Studio Code extension, model Claude Opus 5.5)

Technologies: JavaScript (ES modules), Node.js, Express 5, SQLite (better-sqlite3), HTML5, CSS3, vanilla JavaScript, `node:test`

GitHub Repository:
https://github.com/valentinaAnt/expense-tracker-budget-analyzer

Submission Date: October 26, 2026

Academic Project

Contents

1 Project Idea and System Requirements **1**
1.1 Project Overview . . . 1
1.2 Functional Requirements . . . 1
1.3 Non-Functional Requirements . . . 1
1.4 Scope and Expected Outcome . . . 1

2 System Architecture – Modules **2**
2.1 Application Structure . . . 2
2.2 User Interface Module . . . 2
2.3 Expense Management Module . . . 2
2.4 Database Module . . . 2
2.5 Budget Management and Analytics Module . . . 3
2.6 Testing Module . . . 3

3 Development Process per Module **3**
3.1 User Interface Development . . . 4
3.2 Expense Management Development . . . 4
3.3 Database Development . . . 5
3.4 Budget and Analytics Development . . . 5
3.5 Testing Development . . . 6

4 Challenges and Tool Comparison **6**
4.1 Biggest Challenges . . . 6
4.2 AI Tool Comparison . . . 7
4.3 Most Helpful AI Tool . . . 7
4.4 Possible Improvements . . . 7

5 Working System Evidence **7**
5.1 Screenshot 1: Main Page in Light and Dark Theme . . . 8
5.2 Screenshot 2: Automated Test Results . . . 9

6 Repository and Conclusion **9**
6.1 GitHub Repository . . . 9
6.2 Conclusion . . . 9

1 Project Idea and System Requirements

1.1 Project Overview

The **Expense Tracker & Budget Analyzer** is a browser-based personal finance application for one person. The user records expenses (amount, category, date, description), sets a monthly spending limit per category, and reviews a dashboard that shows the month's spending, its distribution across categories, how much of each budget is used, and the most expensive categories. It addresses a common problem: overspending is often noticed only at the end of the month, and spreadsheets give no warning when a budget is nearly used up. The application runs locally with a Node.js and Express backend, a SQLite database, and a plain HTML, CSS, and JavaScript interface, and was developed incrementally with Claude Code as the AI coding assistant.

1.2 Functional Requirements

1. **Expense management:** add, view, edit, and delete (with confirmation) expenses with an amount, category, date, and description, and export the listed expenses as a CSV file.
2. **Categories and filtering:** nine default categories; users can add their own, rename them, and delete them (expenses of a deleted category are moved to a category the user chooses). Expenses can be filtered by month and by category.
3. **Financial dashboard:** for a selected month, total spending, total budget, budget left or exceeded, and the number of categories over budget.
4. **Budget management:** one recurring monthly limit per category that can be set, changed, or removed; each category is marked as within budget, close to the limit (from 80 %), or over budget.
5. **Spending analytics:** a bar chart of spending against each budget, the three highest-spending categories with their share of the total, and the spending of the last six months, all calculated from stored records.
6. **Data persistence:** data is stored in SQLite and remains available after the server is restarted.

1.3 Non-Functional Requirements

- **Usability:** responsive layout, labelled fields, keyboard-accessible controls, and messages announced to screen readers; error messages disappear as soon as a field is corrected; a light and a dark theme, chosen by the user and remembered.
- **Reliability:** money is stored as integer cents, so totals are exact; invalid data is rejected with structured JSON errors, and unexpected errors never crash the server.
- **Maintainability:** separate route, service, validation, error-handling, database, and frontend modules, with only two runtime dependencies.
- **Security:** parameterized SQL; user text is rendered with `textContent`, never as HTML, which prevents cross-site scripting; the server listens only on `127.0.0.1`; the database is excluded from Git.
- **Testability:** automated tests on temporary databases, including invalid input, empty months, month boundaries, and budget limits.

1.4 Scope and Expected Outcome

User accounts, bank integration, payments, income tracking, and cloud synchronization are out of scope. The expected outcome is a working, tested application that demonstrates modular design,

persistent data, accurate calculations, and the practical use of an AI coding assistant throughout development.

2 System Architecture – Modules

2.1 Application Structure

The application uses a layered architecture (Figure 1). The browser loads static files served by Express and calls a JSON REST API. Routes only translate HTTP requests into service calls; services contain validation, business rules, and parameterized SQL; the database module opens SQLite and creates the schema. `createApp(db)` receives the database as a parameter, so tests can run the real application against a temporary database.

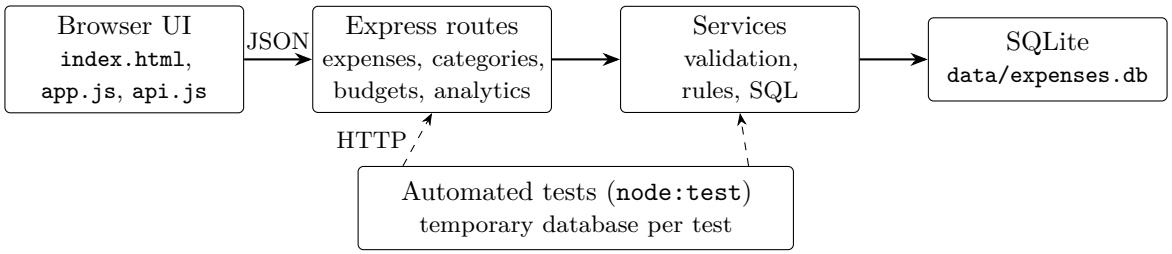


Figure 1: Layered architecture; dashed arrows show how the tests access the system.

Technology choices. Express 5 forwards errors from route handlers to one central error handler. `better-sqlite3` (13.0.3) was chosen over the still experimental built-in `node:sqlite` because it is mature, supports prepared statements and transactions, and ships prebuilt Windows binaries. Tests use the built-in `node:test` runner and `fetch`, so no test libraries are needed. The project targets Node.js 24 LTS; the final tests ran on Node.js 24.22.1 with npm 11.19.1.

2.2 User Interface Module

`public/index.html` is one page in three sections: an overview (summary cards, category bar chart, top categories, six-month trend), expenses (the form with *Clear*/*Cancel* next to the expense list with its category filter and *Export CSV* link), and settings (budgets and categories). `api.js` is the only module that calls `fetch`; `app.js` renders data, handles events, and formats amounts in euros. A `<dialog>` confirms deletions. On phones the expense table turns into cards. The blue-grey design uses colour only for budget warnings and has a light and a dark theme (moon/sun button).

2.3 Expense Management Module

`expenseService.js` validates the whole request, reports all invalid fields together, and performs create, read, update, delete, and filtered list operations. `validation.js` holds shared checks: amounts are parsed as text into integer cents (positive, at most two decimals, up to 1,000,000), dates must be real calendar dates, and IDs positive integers. `csv.js` writes the CSV export and prefixes text starting with `=`, `+`, `-` or `@` so that spreadsheets do not run it as a formula. `errors.js` maps errors to HTTP status codes (Table 1).

2.4 Database Module

`database.js` opens the SQLite file, enables foreign keys, and creates three tables: `categories`, `expenses` (`amount_cents`, `category_id`, `expense_date`, `description`, timestamps), and `budgets` (one per category through a `UNIQUE` constraint). Initialization is repeatable; the default categories

Endpoint	Purpose and responses
`GET /api/expenses`	List, newest first; optional `month` and `categoryId` filters (200; 400)
`GET /api/expenses/export`	The same filtered list as a CSV download (200; 400)
`POST /api/expenses`	Create (201); 400 invalid data or JSON, 413 body too large
`GET, PUT, DELETE /api/expenses/:id`	Read (200), update (200), delete (204); 400 invalid, 404 unknown
`GET, POST /api/categories`	List with expense counts (200), create (201); 400 invalid, 409 duplicate
`PUT, DELETE /api/categories/:id`	Rename (200), delete with its budget (204); optional `moveTo` moves its expenses; 404, 409 duplicate or still in use
`GET /api/budgets`	List all budgets (200)
`PUT, DELETE /api/budgets/:categoryId`	Set or replace (200), remove (204); 400 invalid, 404 unknown
`GET /api/analytics/summary`	Summary for `month` (default: current month); 400 invalid month
`GET /api/analytics/trend`	Spending per month for the last `months` (default 6) and the monthly budget; 400

Table 1: Main API endpoints. Errors use the format `{"error": {code, message, details}}`; unexpected errors return a generic 500.

are inserted only once, tracked with SQLite's `user_version`, so categories deleted by the user are not recreated. A case-insensitive unique index on category names and `CHECK` constraints act as a last line of defence behind the service validation. A category with expenses is deleted only after its expenses are moved to another category, in one transaction.

2.5 Budget Management and Analytics Module

`budgetService.js` stores budgets with one `INSERT ... ON CONFLICT DO UPDATE` statement. `analyticsServic` sums spending per category for the half-open range from the first day of the month to the first day of the next month (the same rule as the expense list) and joins the budgets. The status is *ok* below 80 %, *warning* from 80 % to 100 %, *over* above 100 %, or *no budget*. The trend uses the same month rule and fills months without expenses with zero.

2.6 Testing Module

`tests/` contains unit tests for validation and the services, HTTP integration tests that start the real application on a random port, and tests that run `src/server.js` as a separate process. Every test uses its own temporary database folder, so development data is never modified.

3 Development Process per Module

Workflow. All modules were developed with Claude Code in Visual Studio Code. Claude read and wrote project files and ran terminal commands (tests, server start-up, HTTP requests), so decisions were based on real output rather than assumptions. The work started with a detailed project brief; Claude first produced a plan (requirements, architecture, schema, API, phases) without code, which the student approved. Each phase then followed the same cycle: Claude explained the design, implemented it, ran the tests, and recorded the actual results in `docs/development-log.md`. The key prompts below are quoted verbatim from that project brief; shorter follow-up requests are described in the text and recorded word for word in the development log.

Human review. Nothing was accepted automatically. Every file change and command required the student's approval, each phase was checked in the student's own terminal and browser before it was committed (for example by opening `/api/health` after the first server start), and results that

did not meet expectations, such as the first redesign and the first version of the report prompts, were sent back with concrete feedback. Major decisions (the plan, the currency, the scope of the category feature) were made by the student after Claude presented the options.

3.1 User Interface Development

Approach and Reasoning. A single page without a framework was enough for the scope and avoided a build step. Every chart bar has a text label (for example "€10.20 over budget") so the status does not rely on colour alone.

AI-Assisted Workflow. Claude asked which currency to display, then generated the HTML, CSS, and JavaScript, rendered the page in headless Microsoft Edge, and inspected screenshots at desktop and mobile widths, which revealed a text bug ("1 categories"). At the student's request the interface was later restyled after common dashboard templates; the student rejected the first version's automatic dark mode and asked for the blue-grey palette that became the final design. Later the student found the page poorly arranged; Claude proposed two layouts as mock-ups and the student chose the three-section version. The student then asked for a manual light/dark switch with a moon and a sun, so light stays the default and the browser remembers the choice. Finally, asked what could still be improved, Claude proposed a prioritised list; the student chose all practical items: a category filter, CSV export, moving expenses when a category is deleted, a six-month trend, and expense cards on phones.

Testing Strategy. A temporary script (not in the repository) drove headless Edge through the real interface: validation, adding, editing, deleting, budgets, categories (including moving expenses), the filter and export, the trend, month navigation, a simulated network failure, and the theme switch (including after a page reload). Its final 50 steps all passed, and a description containing `<b>bold?</b>` was shown as plain text, confirming the XSS protection. The student's own use found two problems the script had missed: an error that never disappeared after saving an empty budget, and no way to discard a half-filled expense. Claude reproduced, fixed, and added both to the script; the student then checked the interface again and approved the result.

AI Tool Choice. Claude could check the visible result itself by producing and reading screenshots.

Key Prompts

- Prompt 1 (project brief, FR5): *"Create a clean, responsive interface with: A dashboard with summary cards. An expense form. An expense list with edit and delete actions. A budget management section."*
- Prompt 2 (project brief, FR5): *"The design should be polished enough to demonstrate in screenshots, but prioritize functionality, accessibility, and reliability over decorative complexity."*
- Prompt 3 (project brief, Phase 4): *"Verify that the application works through the browser, not only through isolated code tests."*

3.2 Expense Management Development

Approach and Reasoning. The server validates every request even though the browser does too, because any client can call the API. Amounts are converted to integer cents with a regular expression instead of `parseFloat`, so a value such as `12.345` is rejected rather than rounded.

AI-Assisted Workflow. Claude proposed the split into validation, error handling, services, and thin routes, implemented it, and after the first test run noticed that database failures were not tested, so it added a test that closes the database under a running server.

Testing Strategy. Tests cover the full create–read–update–delete cycle, all invalid fields reported at once, malformed and oversized JSON, invalid and unknown IDs, month boundaries (30 September and 1 November excluded from October), an SQL-injection style description stored as plain text, and

a generic 500 response that keeps the server running. At the end of this phase 49 of 49 tests passed.

AI Tool Choice. Claude ran the tests itself and reacted to the actual output.

Key Prompts

- Prompt 1 (project brief, FR1): *"Validate required fields, positive monetary amounts, dates, and identifiers. Prevent invalid values from reaching the database."*
- Prompt 2 (project brief, FR6): *"Validate input on the server even when the frontend also validates it. Use appropriate HTTP status codes and structured JSON error responses. Handle missing records, malformed requests, invalid amounts, and database errors without crashing the application."*

3.3 Database Development

Approach and Reasoning. Money is stored as integer cents, dates as `YYYY-MM-DD` text so that string order matches date order, and categories in their own table so that expenses and budgets always group by the same identifiers.

AI-Assisted Workflow. The schema was approved in the planning stage; Claude then implemented `openDatabase()`, passed the database to `createApp(db)`, and added a helper that creates a temporary database for every test. Later the student asked how a user could add a missing category. Claude listed the design questions (duplicates, deleting used categories, defaults reappearing on restart) and offered three options; the student chose full add, rename, and delete, which led to the once-only seeding.

Testing Strategy. Tests check that tables and default categories are created once, that foreign key, `CHECK`, and `UNIQUE` violations are rejected, that data is still present after the database is closed and reopened, that deleted defaults stay deleted, and that the student's existing database upgrades without duplicates.

AI Tool Choice. Claude confirmed with `git check-ignore` that the database files are excluded from Git.

Key Prompts

- Prompt 1 (project brief, FR2): *"Store expense and budget records in SQLite. Data must remain available after the server restarts. Use parameterized database queries. Initialize the database schema in a repeatable way and handle database errors appropriately."*
- Prompt 2 (project brief, final instruction): *"For your first response, do not generate application code. Provide: ... 4. A database schema proposal. 5. An API endpoint plan. ... Identify assumptions and wait for my approval before implementation."*

3.4 Budget and Analytics Development

Approach and Reasoning. One SQL query calculates spending per category in cents and joins the budgets. The month rule is shared with the expense list, so the dashboard and the list cannot disagree, and the default month comes from an injectable clock so tests do not depend on the current date.

AI-Assisted Workflow. Claude implemented the services and routes and tried them on a real server with a scratch database: a Food budget of €200 and an expense of €175.25 gave €24.75 remaining, 87.6 % used, status *warning*.

Testing Strategy. The analytics tests use a fixed data set whose results were calculated by hand beforehand, including expenses just outside the month (total €421.19, Food €10.20 over budget). Further tests cover spending exactly equal to the budget, the December–January boundary, and empty and invalid months. After this phase 71 of 71 tests passed.

AI Tool Choice. Claude derived expected totals separately from the implementation and compared them with the actual output.

Key Prompts

- Prompt 1 (project brief, FR3): *"Define clearly how expenses are grouped by month and category. Ensure the budget calculations use consistent date and category rules."*
- Prompt 2 (project brief, FR4): *"Keep analytics accurate and derived from stored records rather than hardcoded sample totals."*

3.5 Testing Development

Approach and Reasoning. The main risks were wrong totals, wrong month boundaries, invalid data reaching the database, and tests that modify real data; hence temporary databases and HTTP tests against the real application.

AI-Assisted Workflow. To prove that the tests detect mistakes, Claude deliberately introduced bugs (month filter `<` changed to `<=`, spending equal to the budget marked *over*, case-sensitive category names, defaults re-added on every start). Each time the expected tests failed; the change was reverted and the suite passed again. A coverage report also revealed a real bug: `PORT=0` silently fell back to port 3000. When the fix was undone on purpose, one new test still passed and another hung, so both were strengthened until they failed on the buggy code.

Testing Strategy. The final suite has 124 tests in 26 groups, including CSV formatting, moving expenses between categories, the trend, and tests that start `src/server.js` as a real process and check that data survives a restart. The final run reported 124 passed and 0 failed, with 97.6 % line and 97.3 % branch coverage (the uncovered code is mainly the Ctrl+C shutdown handler). The same deliberate-bug check was repeated for the new features: a trend window one month off and a deletion that skipped moving the expenses were both caught. As a last step the committed project was cloned into an empty folder and set up as the README describes: all 124 tests passed, and the server answered `/api/health` with `{"status":"ok","database":"ok"}`, confirming that nothing needed to run the project was missing from Git. This check also revealed a new high-severity advisory for `body-parser`, which Express uses to read JSON (an earlier check had reported 0 vulnerabilities); `npm audit fix` updated it from 2.3.0 to 2.4.0, after which `npm audit` reported 0 vulnerabilities and all tests passed again.

AI Tool Choice. Claude executed the tests and reported failures instead of claiming success.

Key Prompts

- Prompt 1 (project brief, FR7): *"Use a temporary or isolated test database so that tests do not destroy or alter real development data."*
- Prompt 2 (project brief, FR7): *"Do not claim that tests pass until they have actually been executed and their output has been reviewed."*

4 Challenges and Tool Comparison

4.1 Biggest Challenges

- **Windows set-up:** PowerShell blocked `npm.ps1`; Claude inspected the execution policy and proposed a per-user `RemoteSigned` policy or `npm.cmd`.
- **Git and GitHub:** commits appeared under a different GitHub account because Git had derived the work email; a repository-specific identity with the GitHub no-reply address fixed it. A later push was rejected ("stale info") because the GitHub repository had been recreated empty; `git fetch -prune` resolved it.

- **Switching to Node.js 24 LTS:** Claude suggested `winget` without checking that Node was managed by `nvm-windows`, which broke `npm`. Error logs revealed the version manager, wrong environment variables were removed, and `nvm use 24.22.1` restored a clean installation. A project `.npmrc` (`ignore-scripts=true`) then stopped npm from trying to compile better-sqlite3, which ships prebuilt binaries.
- **Problems only a person noticed:** all automated checks passed, yet the student found a lingering error message and no way to clear the expense form. While fixing them, Claude found its own accessibility bug: invalid fields had an empty `aria-invalid` attribute, which screen readers ignore.
- **Calculation accuracy:** floating-point money and month boundaries were handled by integer cents, one half-open month rule, and boundary tests.

4.2 AI Tool Comparison

Only one AI tool, Claude Code, was used, so no first-hand comparison with other tools can be made. Its strengths were access to the real project context (files, terminal output, browser screenshots), explanations before each change, and verification through tests and deliberate fault injection. Its limitations were also visible: a suggested PowerShell command for multi-line commit messages did not work, a piped commit message added an invisible byte-order mark, the `winget` suggestion was made without checking the environment, and its first redesign did not match the student's taste. Every result therefore still had to be checked and directed.

4.3 Most Helpful AI Tool

Claude Code was the most helpful, and only, AI tool. It was most useful in diagnosing environment problems from real output and in testing: it calculated expected results independently and proved, by introducing deliberate bugs, that the tests detect wrong month boundaries and wrong budget status.

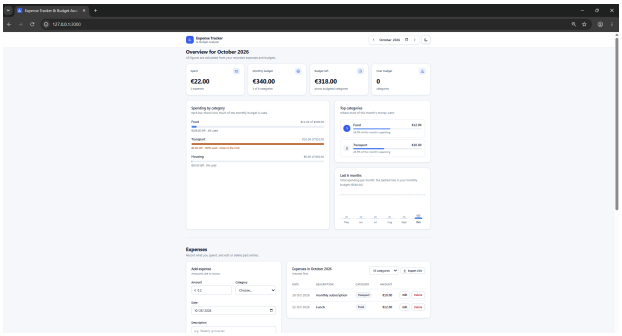
4.4 Possible Improvements

Income tracking, budgets that differ per month, and automated browser tests included in the repository. (Moving expenses when deleting a category, a multi-month trend, and CSV export were listed here earlier and have since been built.)

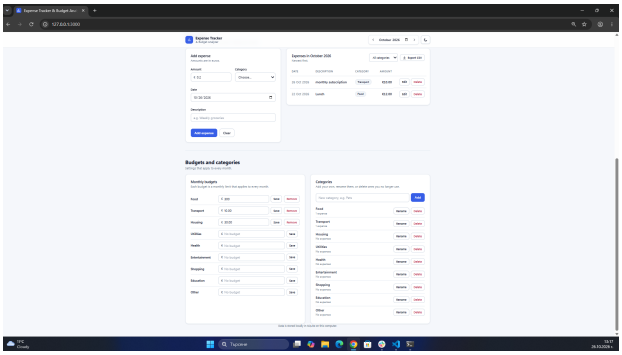
5 Working System Evidence

The following screenshots were taken from the running application and the test suite on the student's computer.

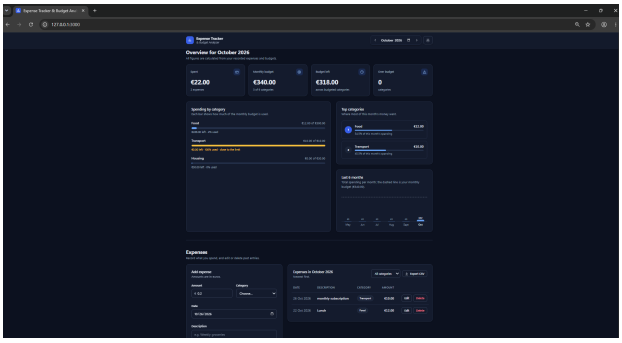
5.1 Screenshot 1: Main Page in Light and Dark Theme



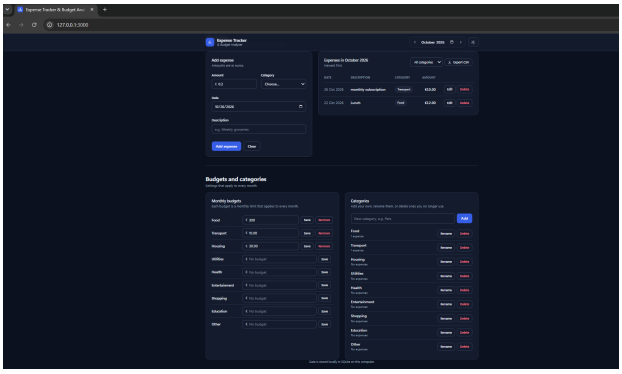
(a) Light theme, top: overview cards, spending by category, top categories, the last 6 months and the start of *Expenses*.



(b) Light theme, bottom: the expense form and list, the monthly budgets and the categories.



(c) Dark theme, top (the same page after clicking the moon button).



(d) Dark theme, bottom.

Figure 2: Main page for October 2026 (spent €22.00 of a €340.00 monthly budget, 0 categories over budget), in the light and the dark theme.

5.2 Screenshot 2: Automated Test Results

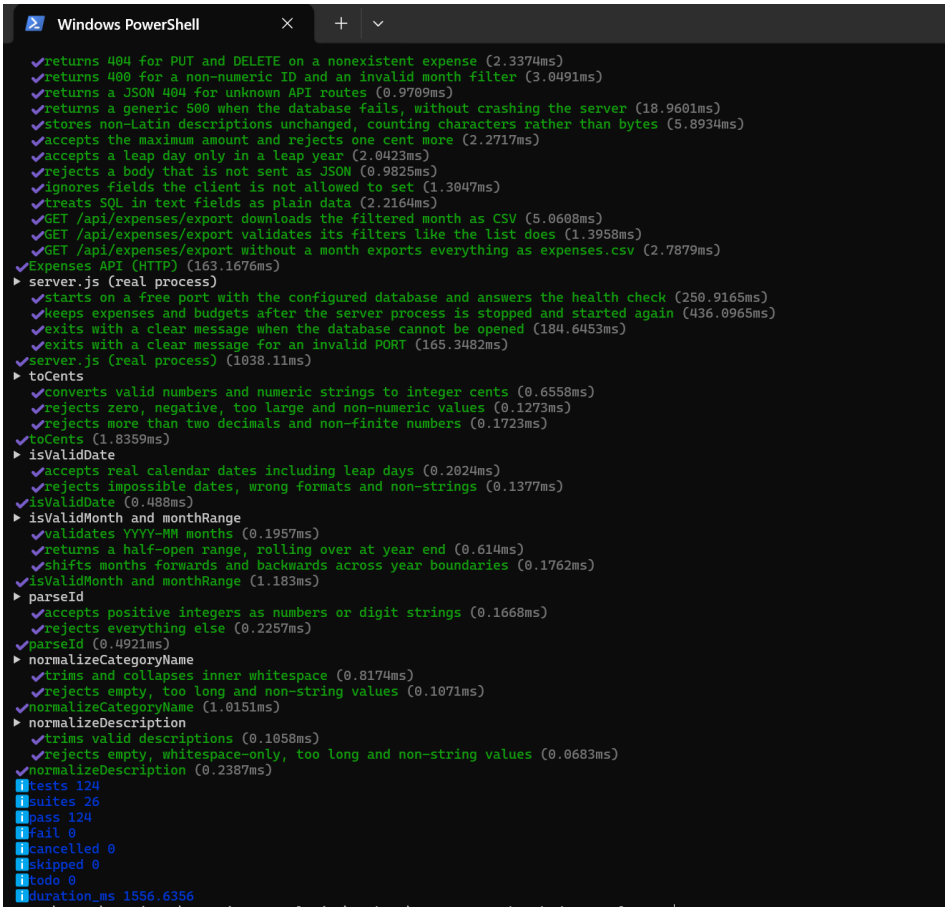


Figure 3: End of the `npm test` output: 124 tests in 26 suites, 124 passed, 0 failed.

6 Repository and Conclusion

6.1 GitHub Repository

The source code, tests, development log, and this report are available at https://github.com/valentinaAnt/expense-tracker-budget-analyzer.

6.2 Conclusion

The completed system meets all functional requirements: a user can record, edit, filter, export, and delete expenses, manage categories, set monthly budgets, and see accurate totals, budget usage, top categories, and a six-month trend on a responsive dashboard, with all data stored in SQLite. 124 automated tests, a 50-step browser check, the student's own use of the interface, and a set-up from a clean copy of the repository confirmed that it works. Its main limitations are the deliberate scope boundaries (one user, one currency, no income) and browser tests that are not yet part of the repository.

The project showed that an AI coding assistant is most effective when the work is divided into small, approved steps and every claim is checked against real output. Planning before coding, keeping a development log, and asking the assistant to prove that its tests can fail were more valuable than generating large amounts of code quickly.